DJANGO ON LINUX · TUTORIAL SERIES

End-to-End Django Tutorial on Linux

From a blank terminal to a deployed Student Management System

24 steps | 1 real project | Beginner friendly

Terminal

```
$ python3 -m venv venv
$ source venv/bin/activate
(venv) $ pip install django
(venv) $ django-admin --version
```

What is Django?

Django is a free, open-source web framework written in Python. It gives you the parts every web app needs, so you can focus on your idea.

Born in a newsroom in 2003 and released as open source in 2005.

Python-based

Write your whole backend in readable, well-documented Python.

Batteries included

Authentication, an admin site, forms and an ORM ship in the box.

Open source

Free under the BSD license and run by the Django Software Foundation.

Why developers use Django

Fast to build

Ready-made parts let you ship in days, not months.

Powerful ORM

Query your database with Python instead of raw SQL.

Secure by default

Built-in guards against CSRF, XSS and SQL injection.

Admin and auth

User login and a ready-made admin site, built in.

Proven at scale

Powers large sites such as Instagram and Mozilla.

Great documentation

Clear official docs and a large, active community.

How Django handles a request

Every page you build in this tutorial follows this path.

1
Browser
`request`
Asks for a page by URL.

→

2
URL router
`urls.py`
Matches the URL to a view.

→

3
View
`views.py`
Runs your Python logic.

→

4
Model + Template
`data + HTML`
Loads data, then fills the page.

→

5
Response
`HTML`
Sent back to the browser.

Django calls this the MVT pattern: Model, View and Template. Step 10 explains it in detail.

MVT ARCHITECTURE

What is the MVT architecture?

MVT stands for Model, View and Template. It is the pattern Django uses to organise every web application.

Model

models.py

- Describes your data as Python classes
- One class maps to one database table
- Handles queries, rules and relationships

View

views.py

- Receives the HTTP request
- Runs the logic and asks the model for data
- Returns a response, often a rendered template

Template

templates/*.html

- HTML files written in Django Template Language
- Placeholders and tags insert dynamic data
- Controls how data looks, not how it is fetched

Django itself plays the Controller: its URL dispatcher routes each request to the right view.

MVT ARCHITECTURE

Model: the data layer

- **One class, one table.** Every model is a Python class that subclasses models.Model.
- **Fields are columns.** CharField, IntegerField and EmailField set each column's type and rules.
- **Built-in ORM.** Query with Python, such as Student.objects.filter(), instead of raw SQL.
- **Relationships.** ForeignKey, OneToOneField and ManyToManyField link tables together.
- **Migrations.** makemigrations and migrate turn model changes into database changes.
- **Database independent.** The same code runs on SQLite, PostgreSQL, MySQL and Oracle.
- **Business rules live here.** Validation, defaults and helper methods belong on the model.

models.py

```
from django.db import models

class Student(models.Model):
    name = models.CharField(max_length=100)
    roll_no = models.IntegerField(unique=True)
    email = models.EmailField()

    def __str__(self):
        return self.name
```

Query the table from Python

```
Student.objects.filter(name="Ali")
```

View: the logic layer

- **A misleading name.** A Django view does not draw anything; it decides what to show.
- **Request in, response out.** It receives an HttpRequest and must return an HttpResponse.
- **Talks to the model.** It reads, creates, updates and deletes data through the ORM.
- **Chooses the template.** render() fills a template with a context dictionary.
- **Two styles.** Function-based views are simple; class-based views such as ListView reuse common patterns.
- **More than HTML.** A view can return JSON, a redirect, a file or an error like Http404.
- **Guards access.** Decorators such as login_required protect private pages.

views.py

```
from django.shortcuts import render
from .models import Student

def student_list(request):
    students = Student.objects.all()
    context = {"students": students}
    return render(request, "list.html", context)
```

urls.py

```
path("students/", views.student_list)
```

Template: the presentation layer

- **HTML plus placeholders.** Templates are normal HTML files that use Django Template Language.
- **Variables.** {{ student.name }} prints a value passed in by the view.
- **Tags.** {% for %}, {% if %} and {% url %} add loops, conditions and links.
- **Filters.** {{ name|upper }} changes how a value is displayed.
- **Inheritance.** {% extends %} and {% block %} reuse one layout across many pages.
- **Safe by default.** Variables are auto-escaped, which blocks most XSS attacks.
- **Light logic only.** Heavy Python belongs in views, so designers can edit templates safely.

students/list.html

```
{% extends "base.html" %}
{% block content %}
  <ul>
  {% for s in students %}
    <li>{{ s.name }}</li>
  {% endfor %}
  </ul>
{% endblock %}
```

Store templates in a templates/ folder per app.

MVT ARCHITECTURE

How the three parts work together

1 Browser sends a request

The user opens /students/ and the web server hands the request to Django.

2 Middleware runs

Security, session and authentication checks run before any view code.

3 URL dispatcher matches

urls.py maps the path to a view and captures any parameters.

4 View runs the logic

It validates input, applies rules and asks the model for data.

5 Model queries the database

The ORM turns Python calls into SQL and returns Python objects.

6 Template renders HTML

The context from the view fills the template placeholders.

7 Response goes back

Middleware runs again in reverse, then the browser shows the page.

Why it matters: every page you build follows this same path, so debugging means finding the step that went wrong.

MVT ARCHITECTURE

Why Django separates the layers

Separation of concerns

Data, logic and display each live in their own place, so every file has one job.

Loose coupling

Change a template without touching views, or a model without rewriting HTML.

Team collaboration

Designers edit templates while developers work on models and views at the same time.

Reusability

One model or base template can serve many views and many pages.

Easier testing

Test each layer alone: models with the ORM, views with the test client.

Simpler maintenance

Bugs are easier to find because each symptom points to one layer.

What is MVC?

Model-View-Controller is the classic pattern behind many web frameworks.

Model

data and rules

- Stores the application data
- Contains the business rules
- Knows nothing about the user interface

View

output

- Shows model data to the user
- Can be HTML, JSON or a mobile screen
- Holds little or no business logic

Controller

input and flow

- Receives and interprets user input
- Updates the model, then picks the view
- Coordinates the other two parts

MVC dates back to Smalltalk in the late 1970s and now powers Ruby on Rails, Spring MVC, ASP.NET MVC and Laravel.

Mapping MVC to Django's MVT

MVC part	Django equivalent	What it does in Django
Model	Model	Defines data in models.py and talks to the database through the ORM.
View	Template	Renders the HTML the user sees, from files in the templates folder.
Controller	View + URL dispatcher	urls.py routes the request, then the view function applies the logic.

⚠ Naming trap: Django's View is MVC's Controller, and Django's Template is MVC's View. The Django FAQ explains that the framework itself, through its URL configuration, is the controller.

Key differences at a glance

Aspect	MVC	MVT (Django)
Components	Model, View, Controller	Model, View, Template
Controller role	Written by the developer	Played by the framework's URL dispatcher
Request entry point	A controller action receives the request	A URL pattern routes it to a view
Presentation	The View renders the output	The Template renders the output
Business logic	Controller and Model	View and Model
Request flow	User → Controller → Model → View	User → URL router → View → Model → Template
Code to write	More wiring for routing	Less, because routing is built in
Typical frameworks	Rails, Spring MVC, ASP.NET MVC, Laravel	Django

What the difference means in practice

In a typical MVC framework

- **You write the controllers.** Each action reads input and picks a view.
- **Routing is separate.** A routes file maps URLs to actions.
- **Views can be rich.** View models and components are common.
- **More wiring.** Flexible, but more glue code to maintain.
- **A convention.** Teams must enforce the separation.
- **Common stacks.** Java, C#, PHP and Ruby frameworks.

In Django's MVT

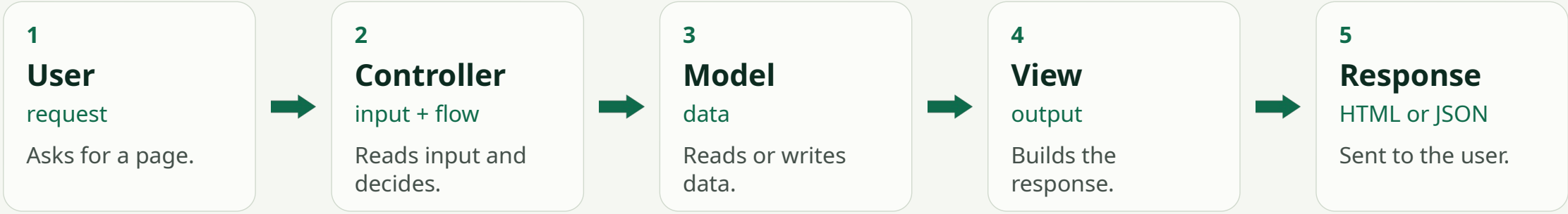
- **The framework is the controller.** No controller classes for routing.
- **One routing file.** urls.py maps every URL to a view.
- **Light templates.** Limited logic keeps the HTML clean.
- **Less glue code.** Admin, auth and forms plug in directly.
- **Convention first.** Any Django developer can read the layout.
- **Plain Python.** Models and views are classes and functions.



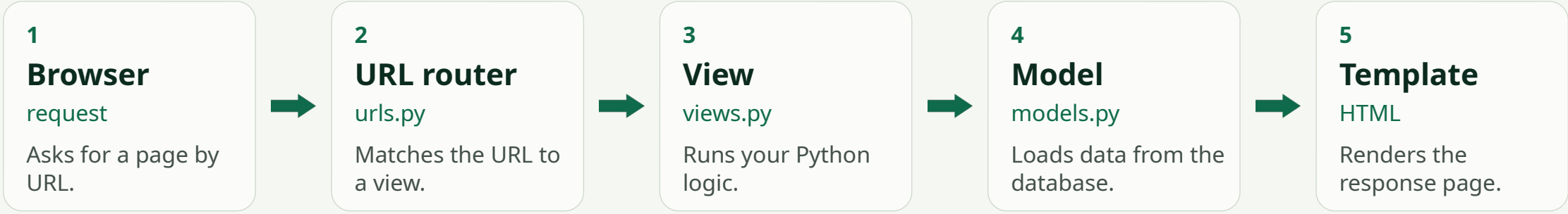
Both patterns separate data, logic and display. The real difference is who writes the controller.

The same request, two patterns

MVC A controller you write sits in the middle of the request.



MVT Django's URL router is the controller, so your code starts at the view.



WHY DJANGO

Why use Django: build faster

Rapid development

Ready-made components take you from idea to working app in days, not months.

DRY principle

Define things once and reuse apps, templates and models across the project.

Batteries included

Authentication, sessions, forms, caching, email and i18n ship with the framework.

Clean URL design

Readable, flexible URL patterns with no file extensions or query clutter.

Automatic admin site

A full create, read, update and delete interface is generated from your models.

Built-in testing tools

A test client and test runner make automated tests part of everyday work.

Powerful ORM

Write Python instead of SQL; migrations track every change to the schema.

Excellent documentation

Detailed official docs and tutorials, plus a large and helpful community.

WHY DJANGO

Why use Django: stay secure at scale

Secure by default
Built-in defences against SQL injection, XSS, CSRF and clickjacking.

Caching framework
Cache whole sites, single views or fragments in Memcached, Redis or a database.

Safe password storage
Passwords are salted and hashed, with PBKDF2 as the default algorithm.

Async support
ASGI and async views handle many concurrent connections efficiently.

Deployment checks
manage.py check --deploy flags risky settings before you go live.

Multiple databases
PostgreSQL, MySQL, MariaDB, SQLite and Oracle work with the same model code.

Scales horizontally
A shared-nothing design lets you add servers behind a load balancer.

Long-term support
LTS releases such as Django 5.2 receive security fixes until April 2028.

WHY DJANGO

Why use Django: ecosystem and fit

Python ecosystem

Reuse pandas, NumPy and machine-learning libraries in the same codebase.

Rich packages

Django REST framework, Celery, Channels and Wagtail extend the core.

Active community

The Django Software Foundation maintains it with regular, well-documented releases.

Django is a great fit for

- Content-heavy and data-driven websites
- Dashboards and internal tools
- E-commerce, social and marketplace platforms
- REST APIs with Django REST framework

Consider a lighter tool when

- You need one tiny, single-endpoint service
- You want to choose every component yourself
- Real-time is the whole product (add Channels)
- Your team has no Python experience yet

DJANGO IN THE WILD

Companies that use Django

Engineering write-ups and public repositories show Django behind these products.

Instagram

social media

Its engineers describe one of the largest Django deployments in the world.

Pinterest

social discovery

Built its early product on Django before its stack grew more specialised.

Mozilla

open source

The add-ons site addons.mozilla.org is a Django app and API.

Disqus

commenting

Scaled a Django app to millions of comments and heavy traffic.

Eventbrite

events and tickets

Runs ticketing on Django and has shared lessons from scaling it.

Sentry

developer tools

The open-source error tracker is a Python and Django application.

The Washington Post

news media

Uses Django in parts of its online publishing infrastructure.

The Onion

satire and news

Moved to Django for easy content updates and a secure backend.

Usage can be partial or historical. Check each company's engineering blog for current details.

DJANGO IN THE WILD

Django across industries

Industry	Examples	Why Django fits
Social media	Instagram, Pinterest	Fast iteration and an ORM that handles relational data at scale.
News and publishing	The Washington Post, The Onion	The admin site works as a ready-made editorial tool.
Developer tools	Sentry, Disqus	Python libraries and heavy event or comment traffic.
Events and commerce	Eventbrite	Transactions, accounts and permissions come built in.
Open source	Mozilla	Clear structure that large contributor teams can follow.



Django was born in a newsroom, so content sites and admin tools are still its home ground.

Recap: what you can now explain

What you have now

- The roles of the Model, View and Template
- How a request travels through Django
- Why Django's URL router acts as the controller
- How MVC terms map onto MVT terms
- Why teams choose Django, and who does

Cheat sheet: MVC → MVT

```
Model       →  Model       models.py
View        →  Template    templates/
Controller  →  View        views.py
               URL router  urls.py
```

Request path
Browser → urls.py → view → model
→ template → browser

You now know the theory. Next, put it to work in your own Django project.

Flask: what it is and how it works

A microframework

Released in 2010 on Werkzeug and Jinja2. The core covers routing, requests and templates only, and a working app fits in a single file.

Unopinionated by design

Django chooses the ORM, auth and forms for you. Flask leaves each choice open and relies on extensions such as Flask-SQLAlchemy, Flask-Login, Flask-WTF and Flask-Migrate.

Structure by convention

Django enforces a project and apps layout with manage.py and settings.py. Flask lets you organise code freely, usually with blueprints, so structure depends on team discipline.

The scaling trade-off

Flask is fast for prototypes and small services. As an app grows you often rebuild what Django already ships, and you must keep extension versions compatible.

Django vs Flask at a glance

Aspect	Django	Flask
Philosophy	Opinionated, batteries included	Minimal core, pick your own parts
Project layout	Enforced project and apps structure	Free-form; blueprints by convention
Routing	Central urls.py with path()	@app.route decorators on functions
ORM	Built-in Django ORM with migrations	SQLAlchemy through an extension
Admin	Automatic admin site	Flask-Admin extension; more setup
Authentication	Users, groups and permissions built in	Flask-Login or Flask-Security, assembled by you
Best size	Medium to large products	Small apps, prototypes and microservices

The same route in both frameworks

Django · urls.py + views.py

```
path("students/", views.student_list)

def student_list(request):
    qs = Student.objects.all()
    return render(request, "list.html", {"students": qs})
```

Flask · app.py

```
@app.route("/students")
def students():
    rows = Student.query.all()
    return render_template("list.html",
                           students=rows)
```

Choose Django when

- The project will grow in features
- You want admin, auth and ORM included
- Many developers need one shared layout

Choose Flask when

- You are prototyping or building a small service
- You want full control of every component
- A tiny footprint matters more than features

What you will build

A complete Student Management System, built step by step.

- **Step 16** — Add, view, edit and delete student records
- **Step 17** — Validate user input with Django forms
- **Step 18** — Manage data from the built-in admin panel
- **Step 19** — Protect pages with login and permissions
- **Step 20** — Style the interface with CSS and static files
- **Steps 21–22** — Test, debug and prepare for deployment

03 · PREREQUISITES

What you should know first

Basic familiarity is enough. You do not need to be an expert in any of these.

Linux

Basic

Move around the terminal and run commands.

Python

Essential

Variables, functions, loops, classes and modules.

HTML and CSS

Basic

Structure a page with tags and style it with CSS.

SQL

Light

Know what tables, rows and simple queries are.

Quick self-check

Skill	You should be able to	Ask yourself
Linux	Navigate folders and manage files	Can you use cd, ls and mkdir?
Python	Write functions, loops and classes	Can you write a class with one method?
HTML	Build pages with forms and links	Can you create a form with an input?
CSS	Style elements with selectors	Can you change a heading's color?
SQL	Read and write rows in tables	Can you explain SELECT and INSERT?

Rusty on one of these? A short refresher is enough. Every Django command is explained as we go.

What you need on your machine

A Linux machine
Ubuntu, Debian, Fedora or a similar distribution

Python 3 and pip
Step 05 shows how to install and check them

A code editor
VS Code, PyCharm or any editor you like

A web browser
To view your site at http://127.0.0.1:8000

Quick check

```
$ cat /etc/os-release
$ python3 --version
$ python3 -m pip --version
```

Missing something? Step 05 covers the setup.

Ready to start?

Before you continue

- I can move around the terminal
- I can read basic Python code
- I know what an HTML page and a CSS file are
- I understand what a database table is
- I have a Linux machine and a browser

UP NEXT · STEP 04

Linux setup

Terminal, package manager, directories and permissions

STEP 04 OF 24

Linux setup

Prepare the terminal, the package manager, your project folder and file permissions.

IN THIS STEP

01 Use the terminal

02 Update with the package manager

03 Create the project folder

04 Understand file permissions

Using the terminal

Reading the prompt

```
student@ubuntu:~/projects$
```

student is your user name

ubuntu is the computer's host name

~/projects is the folder you are in

$ means ready for a command (# means root)

Command	What it does
pwd	Print the folder you are in
ls -la	List all files with details
cd folder	Move into a folder
cd ..	Go up one level
mkdir name	Create a new folder
touch file	Create an empty file
clear	Clear the screen

Updating with the package manager

1 Refresh the package list

```
sudo apt update
```

Fetches what is available. Nothing is installed yet.

2 Upgrade installed software

```
sudo apt upgrade -y
```

Installs newer versions. -y answers yes for you.

3 Install helpful tools

```
sudo apt install -y git curl tree
```

git tracks code, curl downloads, tree draws folders.

OTHER DISTRIBUTIONS

Fedora

```
sudo dnf upgrade
sudo dnf install -y git curl tree
```

Arch Linux

```
sudo pacman -Syu
sudo pacman -S git curl tree
```

⚠ Run the update first, or apt may report “Unable to locate package”.

Creating the project folder

1 `cd ~`

Go to your home folder (plain cd does the same).

2 `mkdir -p ~/projects/student_management`

Create both folders; -p builds any missing parent.

3 `cd ~/projects/student_management`

Move into the new project folder.

4 `pwd`

Print your location to confirm it worked.

Result

```
/home/student
└── projects
    └── student_management
```

Your Django project will live in **student_management**. Step 05 adds the virtual environment inside it.

Understanding file permissions

```
$ ls -l
-rw-r--r-- 1 student student 120 Oct 22 notes.txt
drwxr-xr-x 2 student student 4096 Oct 22 projects
-rwxr-xr-x 1 student student 58 Oct 22 run.sh
```

Reading the permissions of run.sh: -rwxr-xr-x

-	rwx	r-x	r-x
Type	**Owner**	**Group**	**Others**

r = read, w = write, x = execute, - = not allowed
First character: - is a file, d is a folder

How chmod numbers work

```
r = 4 · w = 2 · x = 1
chmod 755 run.sh
7 = 4+2+1 = rwx → owner
5 = 4+0+1 = r-x → group
5 = 4+0+1 = r-x → others
```

chmod 755	Scripts and folders
chmod 644	Normal files
chmod 600	Private files
chmod +x	Make a script runnable

Checkpoint and common errors

Before you continue

- **cat /etc/os-release** names your distribution
- **sudo apt update** finishes without errors
- **pwd** prints the path to student_management
- **ls -l** shows permissions for each file

Message	What to do
command not found	Check the spelling, or install the tool
Permission denied	Check ls -l, then fix with chmod
Unable to locate package	Run sudo apt update first
No such file or directory	Check the path with pwd and ls

UP NEXT · STEP 05 **Install dependencies** Python, pip and virtual environments

STEP 05 OF 24

Install dependencies

Set up Python 3, pip and an isolated virtual environment for your Django project.

IN THIS STEP

01 Check and install Python 3

02 Manage packages with pip

03 Understand virtual environments

04 Create and activate one

Checking and installing Python 3

1 Check for Python 3

```
python3 --version
```

Prints Python 3.x.y when it is already installed.

2 Install Python, pip and venv

```
sudo apt install -y python3 python3-pip python3-venv
```

python3 is the language, python3-pip adds pip and python3-venv adds virtual environments.

3 Confirm pip works

```
python3 -m pip --version
```

Shows the pip version and the Python it belongs to.

EXPECTED OUTPUT

```
$ python3 --version
Python 3.x.y
```

OTHER DISTRIBUTIONS

Fedora

```
sudo dnf install -y python3 python3-pip
```

Arch Linux

```
sudo pacman -S python python-pip
```

Managing packages with pip

Command	What it does
pip install name	Install a package
pip install name==1.2.3	Install an exact version
pip uninstall name	Remove a package
pip list	List installed packages
pip show name	Show package details
pip freeze	Print names and versions
pip install -r requirements.txt	Install from a list
pip install --upgrade pip	Update pip itself

What is PyPI?

pip downloads packages from PyPI, the Python Package Index at pypi.org.

⚠ Never run sudo pip install. It can break tools your system depends on.

pip install django and pip install Django are the same: names are not case-sensitive.

Why virtual environments matter

Without a virtual environment

```
one shared Python
Project A needs Django 4.2
Project B needs Django 5.2
```

Only one Django version can be installed, so one project breaks.

With a virtual environment

```
venv of Project A
Django 4.2
```

```
venv of Project B
Django 5.2
```

Each project keeps its own packages and versions.

⚠ Newer Ubuntu and Debian releases block global pip installs with the error: externally-managed-environment. A virtual environment is the supported fix.

Creating a virtual environment

1 Open the project folder

```
cd ~/projects/student_management
```

The environment will live inside the project.

2 Create the environment

```
python3 -m venv venv
```

Makes a folder named venv with its own Python and pip.

3 Activate it

```
source venv/bin/activate
```

Turns it on. The prompt now starts with (venv).

AFTER ACTIVATING

```
$ source venv/bin/activate
(venv) $
```

INSIDE VENV

```
student_management/
└── venv/
    ├── bin/ · python, pip, activate
    ├── lib/ · installed packages
    └── pyvenv.cfg · settings
```

Verifying the environment

VERIFY, THEN LEAVE

```
(venv) $ which python
/home/student/projects/student_management/venv/bin/python
(venv) $ python --version
Python 3.x.y
(venv) $ deactivate
$
```

Add venv/ to .gitignore so Git never tracks it.

Common errors

`ensurepip is not available`
Run sudo apt install python3-venv

`externally-managed-environment`
Activate the venv, then run pip

`python: command not found`
Use python3 outside the venv

UP NEXT · STEP 06 **Install Django** — Install and verify Django

STEP 06 OF 24

Install Django

Install Django inside the virtual environment and confirm that it works.

IN THIS STEP

01 Activate the environment

02 Install Django with pip

03 Verify the installation

04 Save the dependencies

Installing Django with pip

1 **Open the project folder**

```
cd ~/projects/student_management
```

Every command in this step runs from here.

2 **Activate the environment**

```
source venv/bin/activate
```

Look for (venv) at the start of the prompt.

3 **Install Django**

```
pip install django
```

Fetches Django and its helpers from PyPI.

EXAMPLE OUTPUT (ABRIDGED)

```
(venv) $ pip install django
Collecting django
Collecting asgiref
Collecting sqlparse
...
Successfully installed asgiref-x.y.z
django-x.y.z sqlparse-x.y.z
```

⚠ No (venv) in the prompt? Activate first, or pip may refuse to install.

Verifying the installation

Command	Prints	What it proves
python -m django --version	x.y.z	Python can import Django
django-admin --version	x.y.z	django-admin runs correctly
pip show django	Package details	pip knows about Django
python -c "import django; print(django.get_version())"	x.y.z	Works from Python code

EXPECTED OUTPUT

```
(venv) $ python -m django --version
x.y.z
```

⚠ Run these inside the activated environment. Outside it you may see “No module named django” or “command not found”.

Saving and recreating dependencies

Package	What it does
Django	The web framework itself
asgiref	Async support library
sqlparse	Parses SQL for Django

AFTER STEP 1

```
student_management/
├── requirements.txt
└── venv/
```

1 Save the packages

```
pip freeze > requirements.txt
```

Writes each package and version to a file.

2 Read the file

```
cat requirements.txt
```

Shows lines such as Django==x.y.z.

3 Recreate it anywhere

```
pip install -r requirements.txt
```

Run in a new venv to get the same versions.

Recap and common errors

What you have now

- Linux updated, tools installed
- Project folder student_management
- Python 3, pip and venv installed
- Virtual environment created, active
- Django installed and verified

Message	What to do
pip: command not found	Activate the venv first
No module named django	Activate, then pip install django
django-admin: command not found	Activate the venv, or reinstall Django
No matching distribution found	Check internet and Python version

UP NEXT · STEP 07 **Create a project** — django-admin startproject